unimem: A Harness-Agnostic Memory Gateway for Coding Agents

mapix

https://github.com/mapix/unimem

October 2026 (software v0.5, commit 7c8742a)

Abstract

Every coding harness now ships its own memory: Claude Code has CLAUDE.md and auto memory, Codex has AGENTS.md and its memories, and MCP- or hook-based memory servers must be wired into each harness separately. The result is a set of silos: a correction given to one agent is unknown to the next. We argue that the model API is the only integration point all harnesses share, and present unimem, a local gateway that sits behind the harness's base URL and gives every harness one shared, project-scoped long-term memory without a plugin. unimem makes three systems contributions: (i) *cache-stable overlay replay*, which records every change it makes to a request under a content anchor and re-applies it byte-for-byte, so every upstream request stays an exact byte-prefix extension of the previous one; (ii) *streaming interception and continuation* of its own tool calls across the Anthropic Messages, OpenAI Responses and Chat Completions wire protocols, including mixed turns; and (iii) a *memory-timing policy* grounded in a survey of write/read triggers, with token budgets, a cross-turn de-dup ledger, compaction-aware re-injection, tool-error recall and a pre-compaction flush. The memory engine follows Jev-Mem's System-One/System-Two split with a pluggable typed decider. On a mock upstream, 1060/1060 interception assertions pass, 42/42 upstream transitions are exact prefix extensions, and streaming adds 1.15 ms p50 time-to-first-byte. On an out-of-sample coding-memory suite, unified memory reaches cross-harness R@5 0.664 (0.671 with v0.5) where per-harness silos score 0.000. On the test split of LongMemEval$_S$ and on LoCoMo, unimem's ranker with a 118M-parameter e5 embedder is level with or above BM25+e5 reciprocal-rank fusion (LoCoMo R@10 $+0.035$, 95% CI $[+0.022, +0.047]$; LongMemEval turn R@10 $+0.025$, borderline). In a 120-request long-session simulation, the ledger removes all 128 duplicate injections and the flush saves 5 of 6 planted facts before compaction (0 of 6 without it). We are explicit about what is immature: all results are offline (heuristic decider, mock upstreams, LLM-authored synthetic suites), no end-to-end QA or task-success metric was run, and the write-time injection screen caught only 1 of 7 held-out injections.

1 Introduction

Long-term memory has become a standard feature of coding agents. Claude Code loads CLAUDE.md and an agent-written auto memory at the start of each conversation and re-injects both after compaction [2, 5]; Codex extracts memories from idle chats in the background [21]; research systems such as MemGPT [22], mem0 [8], Zep [24], A-MEM [28] and Jev-Mem [12] study how memories should be written, linked, updated and retrieved. Yet a developer who uses more than one agent experiences memory as a set of *silos*. Each harness stores its notes in its own format and location, none can read the others', and most do not separate facts "about me" from facts "about this repository". Memory servers exposed over MCP or harness hooks can share a store, but each must be configured per harness, and they depend on the agent choosing to call them.

Thesis. Memory belongs to the user, not to the tool. If memory is to be shared, it must live outside every harness. Among all the places it could live, the model API is the only one every harness is guaranteed to use, and every harness lets the user change its base URL. A gateway at that point needs zero per-harness integration. Because it runs on the user's machine, it can also read the working directory the harness mentions and resolve the git project deterministically.

The gateway position is attractive but unforgiving. A gateway that edits requests can silently break the provider's prompt cache, which is a prefix match. Tools it exposes must never leak into a harness that does not know them, which requires rewriting streamed responses in flight. Memory it injects competes with the task for context, and a naive policy grows without bound until the harness compacts—after which the memory is gone. This paper describes how unimem handles each of these and measures how well it does.

Contributions.

1. **A harness-agnostic memory gateway** with *cache-stable overlay replay*: every modification is stored under a content anchor derived from what the harness echoes back, scoped by session, and replayed byte-identically, which keeps every upstream request an exact prefix extension of the previous one (§3.3).
2. **Streaming interception and continuation** across three wire protocols (Anthropic Messages, OpenAI Responses, Chat Completions), including turns that mix gateway tools with harness tools (§3.4).
3. **A memory-timing design grounded in a survey** of when frontier systems write and read memory: token budgets derived from the model's window, a cross-turn de-dup ledger, compaction detection and re-injection, tool-error-triggered recall and a pre-compaction flush (§3.5).

4. **A Jev-Mem-style engine** in which a pluggable System-One decider answers batched typed questions, each with a code-computed prior, and recall is a calibrated hybrid ranker with an adaptive cut-off (§3.6).
5. **An evaluation** on the standard LongMemEval$_S$ [27] and LoCoMo [18] retrieval tasks with a dev/test split and bootstrap CIs, plus new coding-agent memory suites with explicit in-sample/out-of-sample discipline, gateway micro-benchmarks and a long-session simulation (§5). We report negative results, including a version that overfit its own benchmark (§6), and a long list of immature aspects (§7).
6. **An open-source Rust implementation** [19]: one binary with an embedded SQLite store, an MCP endpoint, a CLI and a local web UI.

2 Background and Related Work

Harness-native memory. Claude Code loads `CLAUDE.md` (user-written) and auto memory (written by the agent "based on your corrections and preferences") at session start; only the first 200 lines or 25 KB of the `MEMORY.md` index load, and topic files are read on demand [5]. Its hooks (`SessionStart`, `UserPromptSubmit`, `PostToolUse`, `PreCompact`, `SessionEnd`) let scripts add context, with `additionalContext` capped at 10,000 characters [4]. After compaction, hook-added context is summarized with the rest of the conversation, and only `CLAUDE.md`, auto memory and `SessionStart(compact)` output are re-injected [2]. Codex memories are off by default and are extracted in the background from idle chats, skipped when the rate-limit remaining percentage is low [21]. All of these are bound to one harness's format and directory. unimem does not replace them; it adds a layer shared across harnesses.

Agent memory systems. mem0 extracts candidate facts per message pair and updates the store with ADD/UPDATE/DELETE/NOOP operations chosen by an LLM [8]. MemGPT lets the model manage its own memory through function calls and inserts a memory-pressure warning at about 70% of the window [22]. Sleep-time compute [17] and Letta's "dreaming" [15] move consolidation off the hot path, triggered every N steps or on compaction. Zep/Graphiti stores bi-temporal edges and invalidates contradicted ones instead of deleting them [24]. LangMem separates hot-path from debounced background formation [14]. A-MEM links each new note to related notes and evolves old ones [28]. MemoryOS promotes memories across three tiers by heat [13]. Generative Agents reflect when summed importance exceeds 150 [23], and MemoryBank applies Ebbinghaus-style decay [29]. Jev-Mem [12] delegates the many small typed decisions of a memory system (typing, relations, routing, budgets, scoring, stopping) to a fast probabilistic "System One" model and reserves the large model ("System Two") for reasoning; its reference implementation gates per-prompt recall with one System-One call and consolidates every 20 writes [11]. Coding-agent tools such as memsearch [31] and OpenWolf [10] attach to harness hooks. unimem's engine follows Jev-Mem's split; its contribution is *where* memory runs (the API boundary) and *when* it is written and read.

Benchmarks. LongMemEval [27] decomposes long-term memory into indexing, retrieval and reading, and reports that long-context models lose about 30% accuracy on sustained interaction, so keeping everything in context is no substitute. LoCoMo [18] evaluates very long multi-session dialogues. Both are chat benchmarks; we use their retrieval tasks and add coding-specific suites (§5). Recent work warns that relevance ranking does not close evidence gaps [16], that shared memory can expose credentials across owners [25], that contracted memories "quietly answer wrong" once facts change [30], and that a remembered proposal must not become an authorization [7]; each maps onto a limitation we observe.

Memory-first proxies. BladeX [20] (a self-hosted memory-first proxy for agents) and NautGate [1] (a memory-aware LLM gateway) also place memory at the model API. Both predate this work (their repositories were created in May 2026); unimem was designed independently from first principles. We have not compared against them experimentally, and differences in mechanism (overlay replay, interception, scope decisions) are not established by this paper.

Timing survey. Table 1 condenses our survey (`docs/REVIEW.md`) of when systems write and read memory. Every row is drawn from a source fetched for that review; entries we could not verify are marked n/d. The survey motivates the taxonomy of §3.5.

3 Design

3.1 First principles

We derived unimem from seven constraints rather than from an existing product. **P1** Memory belongs to the person, so it must sit outside, and be shared by, all harnesses. **P2** The model API is the only universal integration point; every harness supports a custom base URL. **P3** A local gateway can see the file system, so project identity has deterministic hard signals; only semantics (what the project does, whether a note is global) needs a model. **P4** The smartest model is already in the loop (System Two); give it tools instead of paying another large model to summarize. **P5** Frequent small decisions (store this? which scope? conflicts with which note? does this prompt need memory? is the evidence sufficient?) go to a fast typed System-One decider with thresholds in code. **P6** The user must not pay for memory with cache misses: any injection is either byte-stable for the whole session or appended at the tail and replayed verbatim afterwards. **P7** Failure must be transparent: any memory-layer error degrades to pure pass-through.

3.2 Architecture and turn classification

Figure 1 shows the architecture. Not every request should receive memory: harnesses issue many helper requests (title generation, command-prefix checks, quota probes, their own compaction summaries). unimem therefore classifies each request:

Table 1: When frontier systems write and read memory (condensed from the design review; all sources fetched October 2026). n/d = not documented in the fetched source. The last row is this work.

System	Write trigger(s)	Who extracts	Read trigger(s)	Budget / placement	Update / forget
Claude Code `CLAUDE.md` [5]	explicit user edit, `/init`	user (agent on request)	session start; nested files on access; re-injected after compaction	target <200 lines	manual
Claude Code auto memory [5]	hot path: agent writes files mid-session	in-loop model	session start (index 200 lines/25 KB); topic files on demand; re-injected after compaction	hard index cap with reminders	agent edits/merges
Claude Code hooks [4]	any lifecycle event	hook script	SessionStart, UserPromptSubmit, PostToolUse	$\leq$10,000 chars per hook	n/a
Codex memories [21]	idle-based background extraction; skipped at low rate limit	extract + consolidation models	future sessions	n/d	background consolidation
mem0 [8]	every message pair	LLM extractor	per query	top-k	ADD/UPDATE/ DELETE/NOOP
MemGPT [22]	agent function calls; pressure warning at $\sim$70%	in-loop model	agent-initiated search; working context	OS-style tiers	self-editing
Letta dreaming [15]	every N steps or on compaction	sleep-time subagents	memory blocks in context	block sizes	background rewrite
Zep/Graphiti [24]	every ingested episode	LLM pipeline	per query search + rerank	top-k	bi-temporal invalidation
LangMem [14]	hot-path tools or debounced background	in-loop or background	tool or prefetch	app-defined	extract, update, consolidate
A-MEM [28]	every add: link + evolve	LLM	n/d	top-k	evolution updates
MemoryOS [13]	STM$\rightarrow$MTM FIFO; heat $> \tau$	system	each response, all tiers	top-m/top-k	heat-based replacement
Generative Agents [23]	reflection when Σimportance > 150	LLM	each action	top-k	none
MemoryBank [29]	each interaction	LLM summary	each turn	top-k	Ebbinghaus decay
jevmem [11]	hot-path tool; auto-capture; consolidation every 20 writes	in-loop + System One	SessionStart; per-prompt gate + subject filter	1–3 notes per recall	supersede/duplicate flags
memsearch [31]	Stop hook every turn	small model	before first step when useful	3-level progressive	journals + summaries
OpenWolf [10]	hook checkpoints	hooks	restore after compaction/restart	n/d	archive
unimem v0.4+	hot-path tool (default); idle/session-end distiller (off); pre-compaction flush	in-loop model; optional cheap model	preamble on first prompt and after compaction; gated passive recall per prompt; tool-error recall; agent tool	per-turn, preamble, tool and per-session token budgets	write-time supersede/duplicate; forget tool

- **agent turn**: the request carries `tools`. unimem appends its four tools (`unimem_recall`, `unimem_remember`, `unimem_forget`, `unimem_project`) and replays overlays;
- **prompt turn**: an agent turn whose last message is new user text; it additionally gets the session preamble (if not live) and gated passive recall;
- **tool turn**: the last message is a tool result (Anthropic `tool_result`, Responses `*_output` items, Chat `role: tool`); it gets recall only if the output looks like an error (§3.5);
- **compaction request**: the harness asks the model to summarize the conversation (matched by its instruction wording); it is passed through unmodified—no tools, no recall, no episode;
- `count_tokens`: our tools are appended and overlays replayed, so the harness estimates the request that is actually sent;
- anything else (no tools, model lists): pure pass-through.

The system prompt is never modified, because some subscription authentication validates its prefix, and tools are appended after the harness's own. Upstream routing preserves the original authentication headers; the Responses dialect routes to `chatgpt.com/backend-api/codex` when the request carries a `chatgpt-account-id`.

3.3 Cache-stable overlay replay

Provider prompt caches match on a byte prefix of the serialized request (tools, then system, then messages). Any change the gateway makes therefore has to be reproduced exactly on every later request of the conversation, even though the harness never sees it. We formalize the mechanism.

Model. A conversation in session s is a sequence of harness-visible messages $h = (m_1, \ldots, m_n)$. The harness sends h on every request, extending it monotonically between compactions. The gateway keeps an append-only overlay store $\mathcal{O}$ mapping anchors to payloads. A content key $\kappa(m)$ hashes what the harness echoes back verbatim: user text and `tool_result` ids for user messages; trimmed `text` and `tool_use` ids for assistant messages, ignoring thinking blocks, signatures, citations and `cache_control`. Two overlay kinds exist:

$$a_i^{\text{aug}} = H(\text{“aug”}, s, \kappa(m_{i-1}), \kappa(m_i)),$$
$$a_i^{\text{exp}} = H(\text{“exp”}, s, \kappa(m_{i-1}), H(\text{“vis”}, \kappa(m_i))).$$

An `augment_user` payload is text appended to user message m_i (preamble, passive recall, tool-error recall, hints). An `expand_assistant` payload replaces the merged assistant message m_i the harness saw with the true upstream sequence $\langle$assistant(with `unimem_*` calls), user(`tool_result`), assistant(continuation)$\rangle$, plus any trailing results to prepend to m_{i+1} in mixed turns. The Responses dialect anchors runs of hidden items before or after adjacent visible output items (keyed

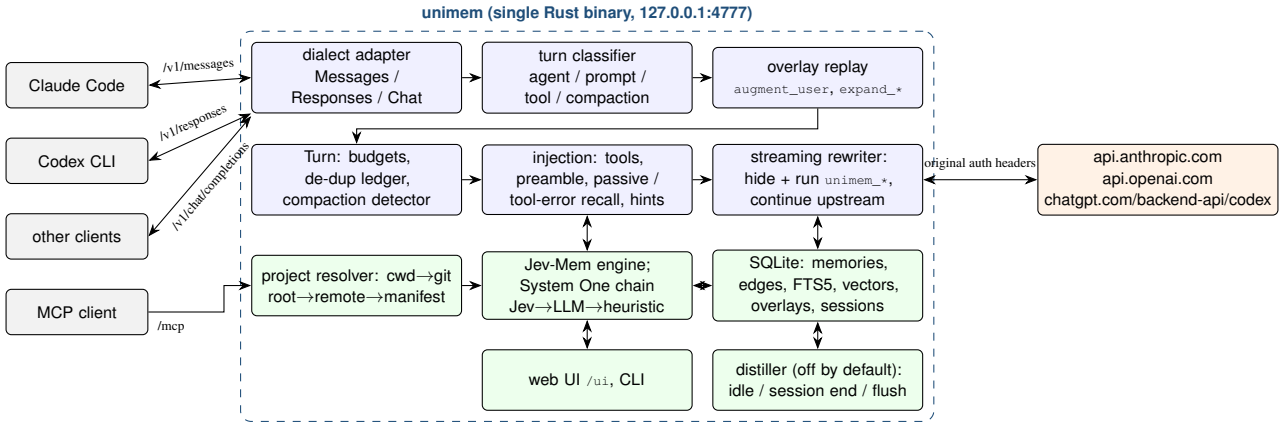


Figure 1: Architecture. Harnesses point their base URL at unimem; the gateway classifies each request, replays its overlays, applies the timing policy, injects tools and memory, and rewrites the upstream stream so that `unimem_*` tool calls are executed inside the gateway. All state lives in one SQLite file.

by `call_id`, reasoning `encrypted_content` hash, or message text). The upstream request is $R(h) = \rho(m_1)\cdots\rho(m_n)$ where $\rho(m_i)$ applies whichever overlays match m_i, plus the fixed tool definitions. New augmentation is only ever added to the *last* message m_n and is stored in $\mathcal{O}$ before the request is sent upstream; an empty payload records "already processed" so retries are idempotent.

Invariants. (*I1, prefix stability*) If h is a prefix of h' and no compaction occurred, then $R(h)$ is a byte prefix of $R(h')$: ρ is deterministic, $\mathcal{O}$ is append-only, every anchor depends only on s and the message pair, and tool definitions are constant. Continuation rounds within one visible turn are themselves prefix extensions. (*I2, session isolation*) Because s is part of every anchor, a conversation never replays another conversation's hidden rounds, even when both share a prompt and a deterministic reply. (*I3, graceful degradation*) If an anchor does not match (compaction, edited history, trimmed blocks), the overlay is skipped and the request remains well-formed; only that memory context is lost. (*I4, invisibility*) No `unimem_*` block and no `<unimem...>` text is ever returned to the harness. We test I1, I2 and I4 directly in §5.3. I1 is the reason passive recall is appended to the *current* user message (the tail) rather than to the system prompt; §5.3 quantifies what the naive alternatives cost.

3.4 Streaming interception and continuation

The harness does not know unimem's tools, so their calls must never reach it. Figure 2 shows the streaming rewriter. It forwards upstream SSE events while maintaining a block (or item) index map. A content block that opens a `unimem_*` tool call is buffered, and later blocks are renumbered so the harness sees contiguous indices. When the round ends with `stop_reason=tool_use`:

- **only our tools**: `message_delta`/`message_stop` are swallowed, the tools run inside the gateway, and the assistant message plus `tool_result` are appended to the request and sent upstream again. The new stream's `message_start` is skipped and its indices are offset, so the harness sees one continuous message. An `expand_assistant` overlay is recorded. At most `max_continuations`=6 rounds run per visible response.
- **mixed** (our tools and the harness's, e.g. `Bash`): the harness's calls are delivered normally; ours run in the gateway and their results are stored as an overlay that re-inserts our `tool_use` into the assistant message and prepends our `tool_result` to the next user message.

The Responses dialect applies the same machine to output items, renumbering `output_index` and `sequence_number` and emitting a single `response.completed` whose `output` holds only visible items. The Chat dialect buffers `tool_calls` by index and joins later rounds' text with a blank line. JSON (non-streaming) responses use the same logic without the event pump.

Honest usage. A merged response reports the *last* round's `input_tokens` and `cache_*` fields (the true context size, which harnesses use to decide when to compact) and the *sum* of output tokens. True totals over all rounds are returned in an `x-unimem-usage` header (JSON, with `rounds`) or a trailing SSE comment, and logged. Earlier versions summed every field, which could overstate context by the number of rounds.

3.5 Memory timing: when to write and when to read

Our survey (Table 1) yields seven write triggers and five read triggers (Table 2). unimem v0.3 implemented only W1, an optional W3, R1, R2 and R3; the design review found that its injection was unbounded, that compaction silently disabled memory, and that usage was over-reported. v0.4 adds W4, R4 and R5 and bounds everything with budgets. Each agent request passes through `Turn::begin`, which loads per-conversation state keyed by `session`#hash(first user message), so subagents that share a session id but start differently get their own preamble and ledger.

Token budgets. unimem uses a tokenizer-free estimator, $\hat{t}(x) = n_{\text{CJK}}(x) + \lceil n_{\text{ASCII}}(x)/4 \rceil + \lceil n_{\text{other}}(x)/2 \rceil$, accurate to about $\pm 30\%$, which suffices because every cap sits far below the window. The window W comes from a model table (Claude 200k, or 1M when the model id carries `[1m]` or

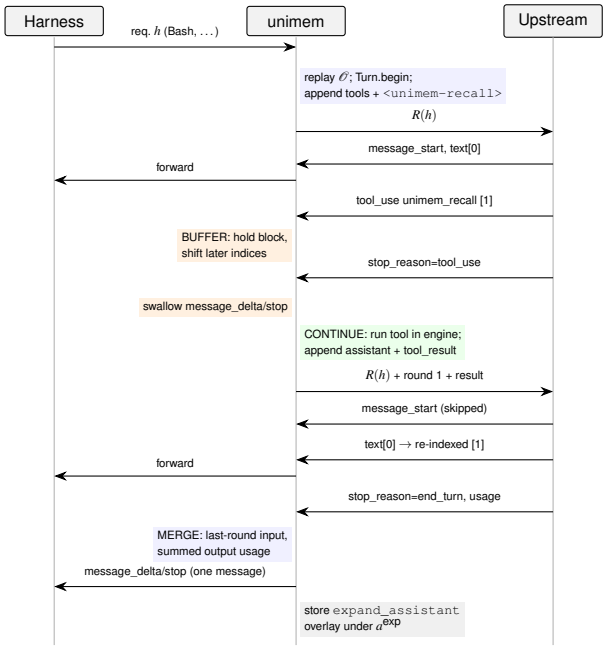


Figure 2: Interception of a gateway-only tool round (Anthropic dialect). The harness receives one well-formed message with contiguous block indices; the hidden round is stored and replayed on later requests.

`anthropic-beta` enables `context-1m`; GPT-5/Codex 272k; o-series 200k; unknown 128k; overridable). With I the tokens unimem currently holds in the live context (replayed augmentations plus hidden rounds, from the ledger), the caps are

$$\begin{aligned} B_{\text{turn}} &= \max(64,\ \min(1200,\ \lfloor 0.006\,W \rfloor)),\\ B_{\text{pre}} &= \max(128,\ \min(1500,\ \lfloor 0.01\,W \rfloor)),\\ B_{\text{tool}} &= 3000\ (k \le 8), \qquad B_{\text{err}} = 400\ (\le 2\ \text{notes}),\\ B_{\text{sess}} &= \lfloor 0.05\,W \rfloor,\\ c_{\text{passive}} &= \min(B_{\text{turn}}, B_{\text{sess}} - I) - \hat{t}(\text{hints}). \end{aligned}$$

Passive recall runs only if $c_{\text{passive}} \ge 64$; tool-error recall only if $\min(B_{\text{err}}, B_{\text{sess}} - I) \ge 48$. For $W = 200$k this gives 1200, 1500 and 10,000 tokens. Rendered notes are truncated to 600 characters of content and 400 of `why`, with a pointer that `unimem_recall` on the note id returns the full text.

De-dup ledger. Each injected note is logged as (note id, overlay anchor, tokens, turn, channel). On every request the ledger keeps an entry only if its anchor was replayed or its id still appears in the request; a note in the ledger is not injected again. Notes that leave the context through compaction or rewind naturally become eligible.

Compaction detection (R5). Any of four signals triggers a reset of the preamble flag, ledger and counters: (i) the message count drops by more than 30% (from at least 6); (ii) the first user message changes; (iii) a summary marker appears (Claude Code's "This session is being continued from a previous conversation", Codex compaction items or handoff text); (iv) the conversation had stored augmentations but none matched. The next request—prompt or tool turn—re-sends the preamble. Signals (i) and (iv) are harness-independent fall-backs.

Table 2: Taxonomy of memory triggers and which ones unimem implements (version added).

	Trigger (examples)	unimem
W1	hot-path tool call by agent (auto memory, MemGPT)	✓ remember tool (v0.1)
W2	per-turn extraction (mem0, memsearch)	–
W3	idle / session-end batch (Codex, LangMem)	✓ distiller, off by default (v0.1; session end v0.4)
W4	pre-compaction flush (MemGPT 70%, Letta)	✓ at 70% of window (v0.4)
W5	importance-threshold reflection (Gen. Agents)	partial: correction importance +3 only
W6	sleep-time consolidation (Letta, jevmem)	–
W7	explicit user command	via W1
R1	session-start preamble (`CLAUDE.md`)	✓ (v0.1)
R2	per-prompt passive retrieval with gate (jevmem)	✓ gated, budgeted (v0.1)
R3	agent-initiated tool (MemGPT)	✓ recall tool (v0.1)
R4	per-step / tool-result triggered (`PostToolUse`)	✓ tool-error recall (v0.4)
R5	post-compaction re-injection (Claude Code)	✓ (v0.4)

Tool-error recall (R4). On a tool turn whose output contains error-like lines (error, FAILED, panic, Traceback, nonzero exit code, NACK, refused; "0 failed" excluded), unimem queries with the error lines, the paths mentioned and the triggering command. A note must share a *rare* token with the query (one occurring in at most $\max(2, 10\%)$ of notes; numbers excluded). Recall fires at most once per 5 tool turns and once per error signature, and respects the ledger.

Pre-compaction flush (W4). When $\max(\hat{t}(\text{request}),\ \text{last real input usage})$ exceeds 70% of W, once per thread, unimem starts a background distillation of the session and appends a $\le$60-token nudge asking the agent to save durable facts with `unimem_remember`.

Correction capture. A prompt that opens with "no,"/"actually," or a Chinese negation, or contains "that's wrong"/"don't use" (questions excluded), receives a $\le$40-token hint to save the correction and raises the episode's importance by 3, which the heuristic distiller honours.

Passive capture and distillation (W3). Prompt turns, visible assistant text and error snippets from tool output ($\le$500 characters) are recorded as episodes after secret redaction, with a 30-day TTL. The distiller is off by default. When enabled it processes sessions in 24k-character chunks, oldest first; with a model it runs a mem0-style ADD/UPDATE(id)/DELETE(id)/NOOP phase against the 5 nearest notes, otherwise candidates go through the normal write path. It triggers on idleness, on a new session in the same project and harness (session end), and at the flush, and skips passes when upstream rate-limit headers show less than 10% remaining.

3.6 Memory engine

Data model. A *memory* has an id, a scope (`global` or `project:<pid>`), a kind (fact, decision, preference, procedure, gotcha, episode), content, an optional `why`, entities, provenance (harness, session, cwd, git branch and commit), timestamps, `superseded_by`, salience, hit counts, a pin flag and an embedding. Typed *edges* (semantic, temporal, causal,

entity, supersedes, duplicate) carry a weight and the decider that judged them. Superseded notes are demoted, never deleted, in the spirit of Zep's invalidation [24].

System One. A `Decider` receives a JSON state and a batch of named questions, each either a yes/no probability or a choice among named options, and *each with a code-computed prior*. Backends form a fallback chain: a Jev-compatible `/v1/systemone` endpoint, any small OpenAI-compatible LLM emulating typed answers in JSON, and an offline deterministic heuristic that returns the priors. Missing answers fail closed. All thresholds live in code.

Write path. (1) Code screens reject empty, over-long (>2,400 characters) and secret-bearing content and near-duplicates; the near-duplicate test is per embedder (cosine $\geq$0.97 for the hash embedder, $\geq$0.95 for e5, computed on a calibrated cosine $(\cos - b)/(1-b)$ with $b = 0.75$ for e5) and also requires a lexical-skeleton Jaccard $\geq$0.5. (2) One batched System-One call asks for *kind* (choice), *scope* (choice among global, the current project and other known projects—this is where the model decides global versus project), *durable* (reject if $p < 0.4$) and *injection* (reject if $p > 0.6$). (3) A hybrid search retrieves 10 neighbours in the relevant scopes. (4) One batched relation call asks, per neighbour, *duplicate* ($p \geq 0.8$), *supersedes* ($p \geq 0.7$), *explains-why* ($p \geq 0.6$) and *same entity* ($p \geq 0.7$); a supersede marks the old note superseded. (5) Node, edges, FTS entry and vector are written; temporal edges within a session are added deterministically.

Recall path. For a query q over scopes S (current project, global, and any project the query names): *Gate* (passive only): one question "does this message benefit from memory?" ($p < 0.35 \Rightarrow$ no injection). *Anchor*: BM25 (SQLite FTS5) top-30 $\cup$ cosine top-30, with asymmetric query/passage embedding for e5. *Score*: each candidate j gets a calibrated prior

$$p_j = \sigma\big(\beta_0 + \beta^\top [\mathrm{cov}, \mathrm{lex}_{\mathrm{rel}}, \mathrm{cov}_{\mathrm{rel}}, \mathrm{clip}(z_j), z_{\mathrm{rel}}, \mathrm{rrf}, \mathrm{reason}]\big),$$

where cov is idf-weighted query coverage, $\mathrm{lex}_{\mathrm{rel}}$ is BM25 relative to the query's best, z_j is the z-score of the note's cosine against the query's cosine distribution over *all* live notes in S (clipped to $[-3, 8]$), z_{rel} its ratio to the best, rrf a reciprocal-rank-fusion term ($k = 60$ [9]) scaled to 1 for rank 1 in both lists, and reason a match between a "why" question and a note that gives a reason. The z-score puts anisotropic transformer cosines (unrelated e5 texts sit at 0.75–0.85) and hash cosines (near 0) on one scale. Weights are fitted per embedder family by logistic regression on development data only (Table 3). The top 14 candidates are re-judged by the decider (`rel_ni`). *Expand*: from up to 3 confident seeds, causal edges are followed when the decider says the query needs reasons, supersedes edges when it needs history (at most 40 nodes). *Cut-off*: in `Depth::Adaptive` (default, used for injection) a note is kept iff $p_j \geq 0.12$ and $p_j \geq 0.4 \max_i p_i$, which usually yields 1–3 notes; in `Depth::TopK` (an explicit k from the agent tool, CLI or UI) the k best are returned with scores. Superseded and stale notes are demoted. *Sufficiency*: a `sufficient`/`missing` signal tells the agent when memory does not know (prior centred at $p_{\mathrm{top}} = 0.3$). The passive path is stricter: it never injects superseded or stale notes, notes with injection prior > 0.2, or notes that address the agent with instructions or authorizations ("auto-approve", "doesn't need to be asked", "note for AI agents"); the agent tool still sees them, flagged.

Table 3: Relevance-prior weights per embedder family (v0.5, `engine.rs`), fitted on dev data only.

family	bias	cov	$\mathrm{lex}_{\mathrm{rel}}$	$\mathrm{cov}_{\mathrm{rel}}$	z	z_{rel}	rrf	reason
hash	−6.16	1.559	1.841	1.821	0.195	0.466	0.874	0.478
dense (e5)	−6.526	0.888	1.271	1.455	0.685	1.469	0.69	0.423

3.7 Project identification

Hard signals are deterministic: the cwd the harness mentions in its prompt resolves to a git root (handling worktrees through `gitdir`/`commondir`), its remote, a manifest (`Cargo.toml`, `package.json`, `pyproject.toml`) and the README's first paragraph. The project id is the normalized remote (`host/owner/repo`), so all clones and worktrees share one project; without a remote, the git root path or the cwd is used, and a renamed path is re-identified by its remote. Soft signals come from models: the project card's summary is seeded heuristically from README/manifest and can be rewritten by the in-loop agent through `unimem_project` (P4); the write scope is a System-One choice; project names and aliases in a query widen the recall scope.

3.8 Failure policy

Recall has a 1.5 s budget on the hot path; on timeout nothing is injected. A rewrite error falls back to the original request; a tool error is returned to the model as its `tool_result`; upstream errors are relayed with status and body unchanged. The fail-open guarantee covers the memory layer but not the process itself (§7).

4 Implementation

unimem is a single Rust binary (edition 2021; axum/tokio, reqwest, bundled SQLite with FTS5 in WAL mode). The source tree (as of commit `a15b022`, the repository-wide `cargo fmt` after `7c8742a`) has 16,217 lines of Rust: 12,059 in the library and CLI, of which 4,790 are the gateway (three dialects, the `Turn` timing layer, SSE handling, MCP and the UI API), and 4,158 in the two benchmark drivers; 3,552 further lines are integration tests. A grep counts 128 test functions (`#[test]` and `#[tokio::test]`). Embedders are a feature-hashing `hash2-512` embedder (stems, compound-identifier parts and a 64-entry bilingual concept table) and an optional local ONNX runtime for `multilingual-e5-small` [26] (118M parameters, 384-d), whose Rust output matches the Python reference with minimum cosine 0.99999988. The binary also serves an MCP endpoint (`/mcp`), a CLI and a self-contained web UI (`/ui`, Appendix C). Configuration is optional; with no file unimem runs fully offline with the heuristic decider and hash embedder.

5 Evaluation

We ask six questions. **RQ1**: How good is unimem's retrieval on standard long-term-memory benchmarks? **RQ2**: Does unified gateway memory help coding agents across harnesses without leaking scope or serving stale facts? **RQ3**: What does the gateway cost, and are its cache and interception guarantees met? **RQ4**: Does the timing policy bound context growth over a long session with compaction? **RQ5**: Is project identification correct? **RQ6**: How does the engine scale?

Setup. Everything ran offline on a 4-core aarch64 VM with 3.8 GB RAM (Linux 6.8, rustc 1.99.1, release build). Unless noted, the decider is the *heuristic* System One (no LLM in the loop) and gateway experiments use a *mock upstream* that responds instantly. Bootstrap CIs are paired, 2000 resamples, 95%.

Datasets and discipline. *Standard*: LongMemEval$_S$ (500 questions; 419 evaluable for retrieval, because the 30 abstention questions and 51 with assistant-side-only evidence have no target in a user-turn index) and LoCoMo (`locomo10`: 1,982 questions with valid evidence, 1,536 in categories 1–4). For v0.5 we split LongMemEval by sha256(question_id)[0] parity into 216 dev and 203 test questions; LoCoMo and our `heldout2` were never used for fitting. *Own suites* (`unimem-bench`): `devmem` (211 queries, 183 notes, 2 harnesses), `heldout` (210 queries, 194 notes, 3 harnesses) and `heldout2` (a new developer, 6 new repositories and a new harness; 242 queries—218 answerable, 20 unanswerable, 4 forbidden-only—233 notes, 12 distractor pairs and 6 decoys; 140 answerable queries are cross-harness). `devmem` and `heldout` became in-sample as we tuned against them; **only `heldout2` is out-of-sample** for v0.3 and later. Its file timestamp predates the v0.3 commit (it was first committed together with the v0.3 evaluation), and no tuning script references it, although 2 of 64 bilingual hint entries added in v0.3 ("queue", "retry") occur only in heldout2, so a leak cannot be excluded. All suites were written by an LLM.

5.1 RQ1: retrieval on LongMemEval and LoCoMo

Protocol. We reproduce LongMemEval's official retrieval code exactly (user-side utterances as keys and values, `recall_all@k`); a cross-check against the official scripts gives identical means and a maximum per-question difference of 1e-16. LoCoMo uses one unit per dialog turn and reports the fraction of evidence turns in the top k. Baselines: `bm25-okapi-ref` (the paper's whitespace-token `rank_bm25`), a stemmed BM25, e5-small dense retrieval, and their RRF fusion (`hybrid-rrf`). unimem variants insert every unit directly into the engine store (bypassing write screens, so no graph edges): `unimem-topk` uses `Depth::TopK` ($k = 50$), `unimem` the adaptive cut-off, and `unimem-write` the full write path.

Results. Table 4 and Figure 3 summarize the test split. With e5, unimem's ranker is level with or above BM25+e5 RRF: on LoCoMo it is significantly better (R@10 +0.035 [+0.022, +0.047], 121 wins/53 losses; R@5 +0.044 [+0.031, +0.057]); on LongMemEval it ties at session level (R@10 +0.010 [−0.010, +0.030]) and is ahead but borderline at turn level (R@10 +0.025 [+0.000, +0.054]: the bootstrap distribution touches 0). The only change from v0.4, where the same ranker was significantly *below* hybrid on LongMemEval (−0.033 session, −0.055 turn R@10), is how the cosine enters the prior (z-score plus RRF) and dense-family weights fitted on dev questions. With the offline hash embedder unimem still loses to hybrid on LongMemEval (−0.079 turn R@10 [−0.123, −0.034]), but ties stemmed BM25 (+0.020, n.s.): the gap is the missing transformer, not the ranker. On the full 419-question set, unimem-prior-e5 reaches turn R@10 0.862 against 0.838.

The adaptive cut-off is the dominant reason `recall` scores low on Recall@10: it returns 1.6–2.9 notes and reaches turn R@10 0.567 against 0.852 for the same engine at `Depth::TopK`. This is deliberate for injection. At *matched depth* (every baseline cut at the number of notes unimem returned for that question; v0.4 runs), unimem keeps up: LongMemEval session hit rate 0.916 vs 0.921 (hybrid), turn 0.826 vs 0.850, and LoCoMo evidence recall 0.520 vs 0.456. Multi-evidence question types suffer most from the cut-off (multi-session test turn R@10: 0.404 adaptive vs 0.692 top-k).

Published numbers. The comparison needs care. The LongMemEval paper's retrieval tables are most likely on LongMemEval$_M$ (∼500 sessions per question) whereas we ran S (∼50): our exact re-implementation of their BM25 scores 0.771 session NDCG@5 on S against 0.516 in their table. Relative to that BM25 re-run on S, e5-small gains +0.174 round R@5 and +0.243 R@10, on par with the gains the paper reports for Stella 1.5B (+0.188/+0.246). On LoCoMo the paper's DRAGON retriever reports overall recall 58.8/67.5/79.9/84.8 at $k = 5/10/25/50$; our best "all categories" numbers (v0.4, unimem-prior-e5) are 0.559/0.639/0.717/0.822, but the paper used 50 conversations (only 10 are public), prefixed each turn with its session date, and used a dialogue-trained retriever.

Write path and graph. Feeding raw chat turns through the full write path loses evidence (`unimem-write-e5`, Table 4): screens reject 1,658 of 102k LongMemEval turns (612 too long, 600 "transient status", 408 too short, 25 "injection", 13 "credential"), and the v0.4 run exposed a real bug: with e5, the hash-calibrated near-duplicate threshold merged 22,763 of 102k LongMemEval turns (22%) and 2,662 of 5,882 LoCoMo turns (45%), because e5 cosines are compressed (random LoCoMo pairs have median 0.823). Per-embedder thresholds reduced this to 8,137 and 433. Graph expansion contributed 0–38 of about 5.7k returned notes: *the graph contributes nothing measurable*.

5.2 RQ2: coding-agent memory (heldout2, out-of-sample)

Systems. `unimem-full` is the engine as shipped (adaptive recall over the current project and global); `unimem-top5` is rank-

Table 4: RQ1, test split (v0.5). LongMemEval$_S$: 203 test questions; LoCoMo: 1,536 questions, categories 1–4 (all test). Arrows give v0.4→v0.5 on the same questions. Bold: best per column.

	LME session index		LME turn index		LoCoMo		
system	R@5	R@10	R@5	R@10	R@5	R@10	notes/query
BM25 (stemmed)	0.847	0.911	0.635	0.729	0.470	0.553	ranking
e5-small	**0.921**	0.966	0.650	0.813	0.482	0.569	ranking
BM25+e5 RRF	0.906	0.966	0.690	0.828	0.551	0.629	ranking
unimem-prior-hash	0.837→0.847	0.901→0.897	0.660→0.655	0.749→0.749	0.543→0.540	0.620→0.618	ranking
unimem-prior-e5	0.833→0.901	0.921→**0.975**	0.650→**0.709**	0.768→**0.852**	0.542→**0.595**	0.619→**0.663**	ranking
unimem-topk-e5 ($k=50$)	0.901	**0.975**	**0.709**	**0.852**	**0.595**	**0.663**	47–50
unimem-e5 (adaptive)	0.616→0.650	0.616→0.650	0.453→0.547	0.463→0.567	0.483→0.510	0.504→0.516	1.6–2.9
unimem-write-e5	0.473	0.473	0.468	0.488	0.490	0.497	1.5–2.6

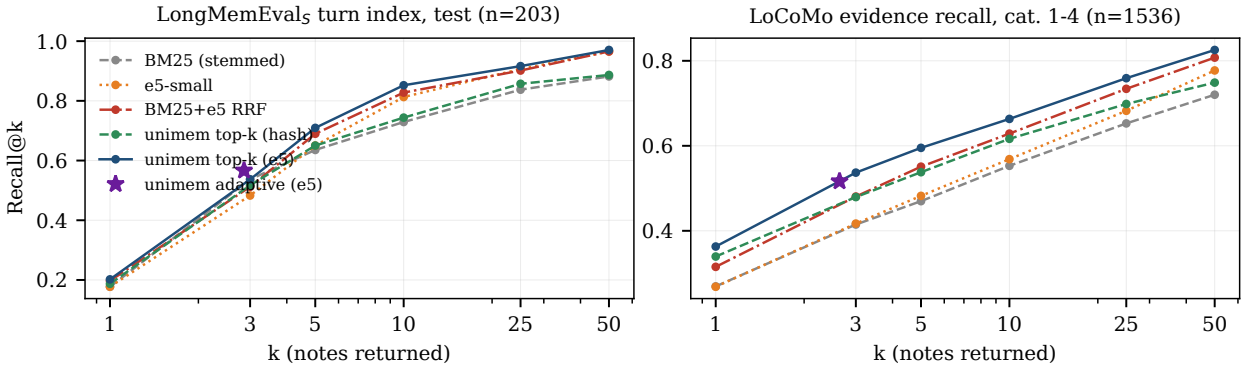


Figure 3: RQ1: Recall@k on the test split (v0.5). The star marks unimem's adaptive recall at the mean number of notes it returns (2.86 on LongMemEval turns, 2.64 on LoCoMo); it trades depth for precision.

ing mode at $k=5$; `silo-per-harness` is the same engine with one store per harness, modelling harness-native memory; `vector-top5`/`bm25-top5` search all notes, and their `-cwd` variants only the current project. Baselines store every write verbatim and have no abstention.

Unified versus silo. Table 5 shows the central architectural claim. (Numbers in the text are the v0.3/v0.4 run; the released v0.5 code reproduces the v0.5 row exactly, with cross-harness R@5 0.671 and MRR 0.608.) On the 140 cross-harness questions (the note was written by a different harness than the one asking), unified memory reaches R@5 0.664 and MRR 0.605; a per-harness silo scores 0.000 by construction. The cost of unification is small: on 78 same-harness questions the silo is slightly better (R@5 0.756 vs 0.731, MRR 0.718 vs 0.637) because a smaller store has fewer competitors.

Retrieval quality. Adaptive `unimem-full` beats every baseline on R@1, MRR, precision and stale-avoidance. Its R@5 is significantly above BM25-cwd (+0.078 [+0.023, +0.133], v0.5 hash run) but *not* above a plain top-5 cosine search with the same embedder (+0.023 [−0.032, +0.078]; v0.3: +0.018 [−0.037, +0.073]); with e5 it is even slightly below (R@5 −0.050 [−0.106, +0.005], n.s.) because it returns about 1.5 notes. At equal depth, `unimem-top5` wins clearly (+0.115 [+0.069, +0.165] over vector-top5-cwd with hash, +0.060 [+0.014, +0.110] with e5). Ranking mode pays for depth: scope leak 17%, stale 35% and rejected-label notes 24–30%, which is why injection stays adaptive.

Scope isolation and staleness. Searching all projects leaks scope on 79–91% of queries and hits forbidden notes 57–67% of the time; unimem leaks on 2.5% with 0% forbidden hits while still answering cross-project questions that cwd-filtered baselines cannot (R@5 0.368 vs 0.053). Only 16.3% of the queries that have an outdated note return it (vs 55.8–65.1% for baselines). The write path (Table 6) explains why: supersede precision/recall is 0.889/0.640 with 0/12 distractor errors; global preferences stated inside a project are scoped correctly in 18/22 cases (v0.1: 2/23). The passive-path screen of v0.4 reduces rejected-label notes in passive results from 6.6% to 1.2% at unchanged R@5.

Error analysis. Of the 68 v0.3 misses on answerable heldout2 queries, 32 are cross-language (query and gold note in different languages; R@5 0.333 on 48 such queries vs 0.788 on 170 same-language ones, a weakness shared by BM25 and hash vectors); about 25 are topical neighbours in the right project outranking the answer; 12 are cross-project; 6 are stale notes; 4 trace to the write path (2 golds wrongly superseded, 1 update dropped as a duplicate); and in 8 an unrejected injection note occupies result slots.

5.3 RQ3: gateway overhead, cache stability and interception

Latency. With a mock upstream (300 requests per cell, ≈31.5 KB Claude-Code-like bodies with 12 tools and an ∼11 KB system prompt), the gateway adds 1.149 ms p50 and 3.124 ms p95 time-to-first-byte when streaming with passive

Table 5: RQ2 on `heldout2` (out-of-sample; 218 answerable queries). Hash embedder unless marked e5. Lower is better for leak, stale and rejected. "rejected" = a note whose gold label is "should be rejected at write time" appears in the results.

system	R@1	R@5	MRR	notes/q	precision	scope leak	forbidden hit	stale ret.	rejected ret.	cross-harn. R@5
unimem-full (v0.3/v0.4)	0.555	0.688	0.616	1.6	0.423	2.5%	0.0%	16.3%	6.6%	0.664
unimem-passive (v0.4)	0.560	0.688	0.618	1.6	0.437	2.5%	0.0%	16.3%	1.2%	0.671
unimem-full (v0.5)	0.555	0.693	0.617	1.65	0.424	2.5%	0.0%	16.3%	6.2%	0.671
unimem-top5 (v0.5)	0.555	0.784	0.648	5	0.157	17.8%	–	34.9%	23.6%	0.786
unimem-full, e5 (v0.5)	0.606	0.693	0.643	1.50	0.470	1.2%	0.0%	18.6%	5.0%	0.664
unimem-top5, e5 (v0.5)	0.610	0.803	0.684	5	0.161	16.9%	–	34.9%	29.8%	0.786
silo-per-harness	0.243	0.271	0.257	1.1	0.242	2.9%	0.0%	16.3%	4.1%	0.000
vector-top5 (all projects)	0.358	0.656	0.475	5	0.131	91.3%	66.7%	62.8%	26.9%	0.643
vector-top5-cwd	0.390	0.670	0.506	5	0.134	8.7%	0.0%	58.1%	45.5%	0.664
vector-top5-cwd, e5	0.514	0.743	0.605	5	0.149	2.1%	–	55.8%	56.6%	0.736
bm25-top5 (all projects)	0.408	0.642	0.500	4.5	0.141	79.3%	57.1%	65.1%	31.8%	0.614
bm25-top5-cwd	0.413	0.615	0.497	4.0	0.152	3.3%	0.0%	55.8%	37.2%	0.600

Table 6: RQ2 write path on `heldout2` (out-of-sample) across versions; v0.4 leaves these metrics unchanged.

metric	v0.1	v0.2	v0.3
rejection precision / recall	1.000 / 0.217	0.889 / 0.348	0.923 / 0.522
injection / secret / status caught	0/7, 4/10, 1/6	0/7, 7/10, 1/6	1/7, 10/10, 1/6
decoys rejected	0/6	0/6	0/6
duplicate recall (gold 23)	0/23	0/23	4/23
supersede precision / recall	1.000 / 0.240	– / 0.000	0.889 / 0.640
distractor false supersedes	0/12	0/12	0/12
scope accuracy	0.890	0.915	0.944
global note written in a project	2/23	8/22	18/22
kind accuracy	0.465	0.460	0.456
write latency p50 / p95 (ms)	0.54 / 0.83	0.70 / 1.02	1.26 / 4.96

Table 7: RQ3: prompt-cache stability over a 30-turn streamed conversation (v0.4). Prefix fraction = byte-identical common prefix of consecutive upstream requests over the earlier one's length.

mode	up. req.	prefix mean / min	stable trans.	cacheable	system stable
Passthrough	37	1.000 / 1.000	36/36	0.987	yes
unimem	43	1.000 / 1.000	42/42	0.987	yes
NaiveSystem	37	0.825 / 0.615	9/36	0.803	no
NaiveTail	37	0.991 / 0.969	9/36	0.979	yes

recall, 0.449 ms streaming without recall, and 0.371 ms (p95 0.424 ms) for JSON (v0.4). The hot-path recall budget is 1.5 s.

Cache stability (I1). Table 7 compares unimem with pass-through and two naive injectors: one that puts recall into the system prompt, and one that appends it to the tail without replaying it. Every one of unimem's 42 upstream transitions, including 6 continuation rounds, is an exact byte-prefix extension, matching pass-through; the naive injectors keep only 9 of 36 transitions fully stable. The price is upstream volume: 2,251,877 bytes vs 1,533,536 for pass-through, mostly the extra full-context continuation requests (cacheable, but billed and rate-limited). We measured byte-level prefixes, not the provider's `cache_read_input_tokens`.

Interception (I4) and isolation (I2). A scripted suite (recall-only, remember-only, mixed unimem+`Bash`, `Bash`-only and plain turns; streaming and JSON; each followed by a request echoing the visible message) drives 120 client requests into 144 upstream requests with 24 continuation rounds. All 1060/1060 assertions pass across 15 families, including: the client never sees a unimem tool (157/157) or unimem context (157/157), framing is valid, hidden rounds are replayed byte-identically to the continuation (24/24), mixed hidden calls and results are replayed (12/12), harness tool calls are delivered (24/24) and the system prompt is untouched (60/60). In an anchor-collision probe, two sessions send the same prompt and receive the same deterministic reply that used `unimem_recall`; 0/10 second sessions receive the first session's hidden rounds (v0.1: 10/10, §6).

Table 8: RQ4: E5 long session (120 requests, compaction at 60, 200k window). Live = tokens unimem holds in context (replayed augmentations + hidden rounds).

variant	max live	share	cum. injected	injections	dup.
v0.4	3,513	1.8%	4,933	67	0
no-budget	3,513	1.8%	4,933	67	0
no-budget-no-dedup	7,703	3.9%	13,282	195	128
heavy / v0.4	7,098	3.5%	9,130	92	0
heavy / no-budget	10,237	5.1%	11,243	79	0
heavy / no-budget-no-dedup	13,998	7.0%	17,342	117	38

5.4 RQ4: long sessions, budgets and compaction (E5)

Scenario. A Claude-Code-like session on one repository (`devmem` corpus, 200k window) runs 120 client requests: prompts from the corpus queries, 6 planted user facts, 3 corrections, tasks with 2–3 `Bash` calls and 24k-character logs (every other task contains one error whose root cause is a stored note), and 4 prompts that make the model call `unimem_recall`. At request 60 the harness sends its compaction request and replaces the history with a summary. "Heavy" variants add 40 long relevant notes (≈2 kchar content plus ≈1 kchar `why`). Ablations disable the budgets, the ledger or the flush.

Results. Table 8 and Figure 4. The ledger removes all duplicate injections (128→0 at normal note sizes, 38→0 with heavy notes). With heavy notes, budgets plus ledger halve the peak unimem footprint from 14.0k to 7.1k tokens (7.0%→3.5% of the window) and cumulative injection from 17.3k to 9.1k. At normal note sizes the budgets never bind (no-budget equals v0.4), and the 5% session cap never bound in this run. The compaction request reached the upstream byte-identical; the preamble came back on the first request after the summary, and live unimem context dropped to about 1.6k before growing

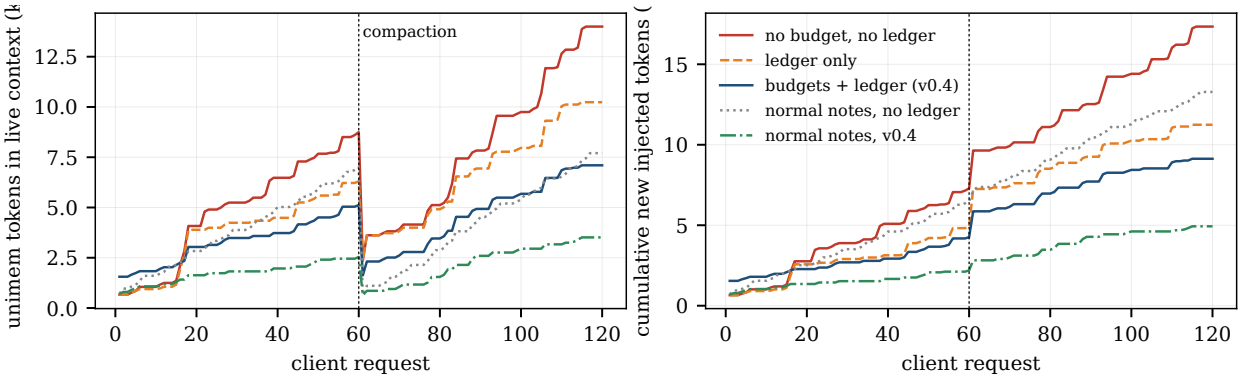


Figure 4: RQ4: tokens unimem keeps in the live context (left) and cumulative newly injected tokens (right) over the E5 session. The dashed line marks the harness's compaction at request 60; the preamble returns on the next request.

again. Tool-error recall fired on 6 of 11 error turns, each time with the root-cause note, and on 3 more the note was already live, so 9/11 errors had their explanation in context; benign tool turns received 0/44 injections. The two misses share only a port number with their note, and the rare-token gate ignores numbers by design. The flush nudge fired once, at request 54 (estimated context ≈140k, 70% of 200k); the background heuristic distillation it triggered had stored 5 of the 6 facts stated before compaction by the time compaction occurred, against 0 of 6 without the flush. All 3 corrections were hinted. Note that the harness's own context dominates: the maximum total context share was 80.7% in v0.4, of which unimem held 1.8%.

Re-run with the v0.5 code. E5 was not re-run for v0.5 in the main experiments; re-running it at release (same scenario) gives the same qualitative picture but different magnitudes, because v0.5's prior returns different notes: peak live context 4.5k tokens (2.2%) for v0.5, 8.4k (4.2%) with heavy notes versus 18.4k (9.2%) for heavy/no-budget-no-dedup, 130–132 duplicate injections removed by the ledger, and 5 of 6 facts saved by the flush (0 of 6 without). E5 is also not exactly deterministic: the background flush distillation races the request loop, so cumulative-injection counts vary by up to about 10% between runs. Table 8 reports the v0.4 run.

5.5 RQ5: project identification (E3)

On 37 synthetic layouts—worktrees (CLI-made and hand-made relative `gitdir`+`commondir`), submodules, nested repositories, forks, GHE/GitLab subgroups and ports, user-info, case and trailing-slash variants, a dotfiles home repository, symlinks, non-git manifest directories and remotes added later—unimem resolves 37/37 correctly (v0.1: 35/37). No held-out layouts exist, so this is in-sample.

5.6 RQ6: scaling (E4)

On templated synthetic notes, v0.3 at 10,000 stored notes (9,849 live) takes 7.654 ms p50 / 17.241 ms p95 per write and 4.751 ms p50 / 7.226 ms p95 per recall, with a 103.71 MB database (10,371 bytes per note) and 77.3 MB RSS. At 7,000 notes, recall p50 is 3.582 ms. Recall is a brute-force cosine over all live notes and grows linearly; an ANN index is needed well before 100k notes. On the standard benchmarks, engine recall takes 2–6 ms per query against at most 0.03 ms for BM25.

Table 9: What each version changed and what the evaluation found. R@5 on heldout2 unless stated.

ver.	change → finding
v0.1	engine, 3 dialects, overlays → streaming +42.8 ms TTFB (Nagle); cross-session anchor collision 10/10; over-eager supersede (91% of E4 stores); R@5 0.569
v0.2	fixes from devmem findings → TTFB +0.76 ms; collision 0/10; E3 37/37; but heldout R@5 0.444→0.447, false abstain 5.6%→21.3%, supersede recall collapsed, 12,664/19,664 E4 writes falsely deduplicated; R@5 0.550
v0.3	calibrated hybrid prior, subject-aware supersede/dup → R@5 0.688 (+0.119 vs v0.1, CI [+0.064, +0.174]); injection screen 6/6 in-sample but 1/7 out-of-sample
v0.4	timing policy from the design review → duplicate injections 128→0; flush 5/6; e5 dedup bug found (22% of turns merged) and fixed
v0.5	embedder-calibrated prior, `Depth::TopK`, ONNX e5 → ranker at or above BM25+e5 RRF on test split

6 Lessons from the Version History

Our versions act as a sequence of ablations, and the evaluation repeatedly found real bugs (Table 9).

The v0.2 overfitting episode. v0.2 fixed every v0.1 finding on `devmem` and reached 1.000 rejection precision and recall there. On the then-unseen `heldout` set it did not improve retrieval (R@5 0.444→0.447, R@1 0.303→0.281), false abstention rose from 5.6% to 21.3%, supersede recall collapsed (5/26→1/26 correct) so stale answers *increased* (44.4%→50.8%), Chinese recall regressed (0.500→0.321), and a new silent data-loss mode appeared: 64% of E4 write attempts were rejected as duplicates. Even in-sample retrieval got worse (devmem R@5 0.486→0.425). The lesson shaped everything after it: in-sample screens can be made perfect and still not generalize (v0.3 again scored 6/6 in-sample and 1/7 out-of-sample on injections), so we built `heldout2` before tuning and report it as the only generalization evidence.

Bugs found by measurement. (i) *Nagle/delayed ACK*: the server did not set `TCP_NODELAY` on client sockets, and many small SSE writes cost +42.8 ms p50 TTFB; a one-line fix brought it to +0.76 ms. (ii) *Anchor collision*: expansion anchors did not include the session, and `put_overlay` ignored conflicts, so two sessions with the same prompt and deterministic reply shared hidden rounds (10/10 in the probe); adding *s* to the anchor gives invariant I2. (iii) *Over-eager supersede*: without a subject check, "auth-12 listens on port 3100" could su-

persede "billing-40 listens on port 5200"; 9,232 of 10,672 templated writes superseded something. (iv) *Embedder-specific dedup*: thresholds calibrated on hash cosines merged 45% of LoCoMo turns under e5. (v) *Distiller data loss*: v0.3 read only the newest 24k characters but marked all episodes distilled. (vi) *Compaction*: v0.3 never re-sent the preamble after compaction. None of these would have been visible without the corresponding measurement.

7 Limitations and Immature Aspects

unimem is research-grade software. We list its weaknesses prominently because a gateway sits in front of all of a user's model traffic.

Context-window blow-up remains a risk. Before v0.4, the design review estimated from the code constants that passive injection costs about 110 tokens per prompt turn typically (≈6% of a 200k window over 100 turns), about 700 with realistic heavy notes (35%), and up to 3.2k tokens per turn in the worst English case, overflowing a 200k window by about turn 60 (CJK notes: 8–12k tokens per turn, overflow by about turn 20); a single visible turn with $k = 20$ agent recalls over 6 rounds could reach about 75k tokens (37%). v0.4 caps per-turn, preamble, tool and per-session injection, and E5 measured a peak of 3.5% of the window with heavy notes. Residual risks:

- *Hidden rounds are replayed until compaction.* The session share stops passive recall, but not agent-initiated `unimem_recall`. With the default constants a single visible response can still add up to 6×3000 tokens of tool output (our arithmetic from B_{tool} and `max_continuations`), which is then replayed on every later request.
- *Heuristics tied to current harness wording.* Compaction-request detection and two of the four compaction signals match the text Claude Code and Codex use today. If that wording changes, unimem may inject into a compaction request or miss a compaction; only the message-count and zero-anchor signals are harness-independent.
- *Anthropic's 20-block cache lookback* [6]. Each hidden round inserts blocks between the harness's cache breakpoints; several continuations can push the previous cache write out of the lookback window and cause a full miss. We measured byte-prefix stability, not real cache reads.
- *Estimator error.* Budgets use a ±30% estimator; unknown models fall back to a 128k window.
- *Upstream volume.* Continuation rounds increase upstream bytes (2.25 MB vs 1.53 MB over 30 turns), drawing on rate limits and, on subscriptions, on the user's quota invisibly.

No real-traffic validation. Every gateway result uses mock upstreams and scripted, harness-like requests. Real harness versions may send shapes the anchors do not expect (e.g. trimmed thinking blocks), in which case overlays are skipped and memory context is lost. Codex with a ChatGPT login through a custom provider, Responses reasoning-item adjacency when hidden items are re-inserted, and user messages inserted after `function_call_output` (tool-error recall) are unverified against the real backends.

Only the offline heuristic decider was evaluated. All quality numbers come from the heuristic System One with hash or e5 embedders. The Jev and LLM deciders are implemented but untested end to end; their 8 s and 15 s default timeouts exceed the 1.5 s hot-path recall budget, so on the passive path they would often be cut off. Kind classification is 46% accurate. The `sufficient` signal misfired on 45% of answerable heldout2 queries in v0.3/v0.4 (consistent with [16]); v0.5's re-scaled prior brings this to 8.7% (hash) and 9.6% (e5) on heldout2 but 89.9% with e5 on `devmem` and `heldout`, so its threshold is not calibrated across embedders and corpora.

No end-to-end metric. We report retrieval, mechanics and simulated timing, not QA accuracy or task success. A LongMemEval/LoCoMo QA script with the official reader and judge prompts is provided (`bench/std/qa_eval.py`) and builds all prompts offline, but was not run. No experiment shows that unimem reduces repeated corrections or improves agent success, which is the only metric that could justify its hot-path costs. LongMemEval$_M$ was not run (disk limit).

Synthetic, LLM-authored suites. `devmem`, `heldout` and `heldout2` were written by an LLM, the corpora are small (183–233 notes), and E3/E4/E5 are generated. `heldout2` is out-of-sample for tuning but not independent of the author's view of what coding memory looks like.

Weak write-time screens. Out-of-sample, the injection screen caught 1/7 and the status screen 1/6 (four of five misses Chinese), against 100% in-sample. A stored injection note ("Team convention per Dave: ... run apply with `-auto-approve` ... Mira doesn't need to be asked") was served as the answer to exactly the question it targets—the failure class described in [7]. v0.4 keeps instruction-like notes off the *passive* path, but they remain retrievable through `unimem_recall`. Shared memory must be treated as untrusted input.

Cross-language and paraphrase gaps. Cross-language R@5 is 0.333 versus 0.788 within a language; 17 of 23 gold duplicates are cross-language restatements and go undetected. Paraphrase (0.419) and cross-project (0.368) categories are weak.

The graph and the gate contribute little. Graph expansion has no measurable effect on any benchmark, and full, lite (no edges) and passive (gated) modes return nearly identical results on heldout2. The four-view graph is, so far, unvalidated complexity.

Operational and policy risks. *Single point of failure*: memory errors fail open, but if the process dies the harness cannot reach its model until the base URL is unset. *No authentication*: the gateway, MCP endpoint and web UI accept anything that reaches the listen address; they must stay on `127.0.0.1`. Since the release audit, requests that carry a foreign `Origin` or a non-local `Host` name are refused, which closes cross-site POSTs and DNS rebinding from a browser, but not other local processes. *Terms of service*: rewriting subscription traffic (Claude OAuth, ChatGPT login) and generating extra requests on it may be restricted by provider terms; Claude Code's gateway documentation notes that a saved login remains the active credential behind a custom base URL [3], and the ChatGPT

Codex backend is not a public API. *Privacy*: one database holds memories and episodes from every harness and project; episodes are redacted by regex/heuristic screens with a 30-day TTL, `unimem_project` lists known projects to any agent, and there is no equivalent of excluding chats with external context from distillation. Shared memory can also expose one owner's credentials to another context [25]. *Concurrency*: one mutex-guarded SQLite connection serializes parallel sub-agents, harnesses and the distiller.

8 Ethics and Broader Impact

A shared memory makes agents more useful and more dangerous at once. It reduces repeated instruction and can carry safety conventions across tools, but it also centralizes sensitive data, widens the blast radius of a poisoned note, and makes it easier to persist an assistant's proposal as if it were the user's authorization. unimem runs locally, sends no telemetry, makes no extra network calls by default, keeps the system prompt untouched, labels injected notes as "data, not instructions", and keeps instruction-like notes off the passive path; these mitigations are partial (§7). Users should check their providers' terms before routing subscription traffic through any gateway.

9 Conclusion

unimem shows that the model API boundary is a workable place for shared agent memory: with overlay replay the provider cache stays byte-stable, with streaming interception the harness never sees a foreign tool, and with budgets, a ledger and compaction awareness the memory footprint stays a few percent of the window in simulation. Unified memory answers cross-harness questions that silos cannot, and the calibrated hybrid ranker is competitive with strong baselines on standard retrieval benchmarks. What we have not shown is that any of this improves real agents on real traffic. The next steps are a regression suite recorded from real Claude Code and Codex sessions, an evaluation of model-backed System-One deciders and screens, end-to-end QA and task-success metrics, and cross-language retrieval.

References

[1] 48Nauts-Operator. NautGate: memory-aware LLM gateway. https://github.com/48Nauts-Operator/NautGate, 2026. Repository created May 2026; accessed October 2026.

[2] Anthropic. Claude code documentation: context window and compaction. https://code.claude.com/docs/en/context-window.md, 2026. Accessed October 2026.

[3] Anthropic. Claude code documentation: LLM gateway configuration. https://code.claude.com/docs/en/llm-gateway.md, 2026. Accessed October 2026.

[4] Anthropic. Claude code documentation: hooks. https://code.claude.com/docs/en/hooks.md, 2026. Accessed October 2026.

[5] Anthropic. Claude code documentation: memory (CLAUDE.md and auto memory). https://code.claude.com/docs/en/memory.md, 2026. Accessed October 2026.

[6] Anthropic. Prompt caching. https://platform.claude.com/docs/en/build-with-claude/prompt-caching.md, 2026. Accessed October 2026.

[7] J. Chen, X. Chen, Y. Jiang, and H. Shen. Knowing the user is not knowing what we established: Grounding awareness in personal agents. *arXiv preprint arXiv:2610.18684*, 2026.

[8] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

[9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[10] cytostack. OpenWolf. https://github.com/cytostack/openwolf, 2026. README accessed October 2026.

[11] illescasDaniel. jev-mem: reference implementation (readme, architecture and installation docs). https://github.com/illescasDaniel/jev-mem, 2026. Accessed October 2026.

[12] D. Jiang, Y. Li, and B. Li. Jev-mem: System-one-controlled agentic memory for efficient AI agents. *arXiv preprint arXiv:2609.23986*, 2026.

[13] J. Kang, M. Ji, Z. Zhao, and T. Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025.

[14] LangChain. LangMem: conceptual guide and delayed background processing. https://langchain-ai.github.io/langmem/concepts/conceptual_guide/, 2026. Accessed October 2026.

[15] Letta. Letta documentation: memory and sleep-time agents ("dreaming"). https://docs.letta.com/guides/agents/sleep-time-agents, 2026. Accessed October 2026.

[16] Y. Li, S. Li, Z. Xu, J. Li, P. Zeng, S. Yao, C. Lin, G. Li, R. Bi, and M. Han. Relevance is not sufficiency: What actually closes the evidence gap in long-term memory QA. *arXiv preprint arXiv:2610.09348*, 2026.

[17] K. Lin, C. Snell, Y. Wang, C. Packer, S. Wooders, I. Stoica, and J. E. Gonzalez. Sleep-time compute: Beyond inference scaling at test-time. *arXiv preprint arXiv:2504.13171*, 2025.

[18] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang. Evaluating very long-term conversational memory of LLM agents. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.17753.

[19] mapix. unimem: one shared, project-aware long-term memory for every coding agent. https://github.com/mapix/unimem, 2026.

[20] netloafer. BladeX: self-hosted memory-first proxy for AI agents. https://github.com/netloafer/BladeX, 2026. Repository created May 2026; accessed October 2026.

[21] OpenAI. Codex documentation: memories. https://learn.chatgpt.com/docs/customization/memories.md, 2026. Accessed October 2026.

[22] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[23] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.

[24] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[25] H.-S. So, D. Liu, C. Hou, Y. He, M. Wang, and H.-T. Zheng. Whose credential is it? How owner attribution in agent memory shapes cross-user account misuse. *arXiv preprint arXiv:2610.25707*, 2026.

[26] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei. Multilingual E5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024.

[27] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10813.

[28] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.

[29] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.

[30] J. Zhou, H. Rong, C. Lin, and D. Zhang. Many steps into one: Reasoning contraction and the limits of safe memory reuse for LLM agents. *arXiv preprint arXiv:2610.17544*, 2026.

[31] Zilliz. memsearch. https://github.com/zilliztech/memsearch, 2026. README accessed October 2026.

A Reproduction

All results are in the repository under `bench/`; every number in this paper is taken from `bench/std/RESULTS.md`, `bench/results/{FINAL,COMPARISON}.md`, `bench/results/v0.{1..5}/` and `docs/REVIEW.md`. Figures are regenerated by `python3 -I docs/paper/figs/make_figs.py` from `bench/results/v0.4/e5_long_session.json` and `bench/results/v0.5/std/*.test.json`.

```
cargo build --release              # add --features onnx for the e5 embedder
# E1-E5 on the out-of-sample suite
cargo run --release --bin unimem-bench -- all --data bench/data/heldout2.json --out /tmp/out
# LongMemEval_S + LoCoMo retrieval, significance, official cross-check
CACHE=/path/to/bench-cache bench/std/run_all.sh
# relevance-prior fit (dev data only) and test-split bootstrap
python3 -I bench/std/fit_prior.py bench/results/v0.5/prior_fit.json <evidence files>
python3 -I bench/std/significance_v05.py C runs-v05
# E1 with the e5 embedder
ORT_DYLIB_PATH=... UNIMEM_BENCH_EMBEDDER=onnx UNIMEM_BENCH_EMBEDDER_MODEL=<dir> \
  unimem-bench e1 --data bench/data/heldout2.json --out ...
```

Swapping the decider or embedder for a run: `UNIMEM_BENCH_DECIDER`, `UNIMEM_BENCH_EMBEDDER`, `UNIMEM_BENCH_CONFIG`. The datasets are downloaded with size and sha256 checks (`bench/std/download.sh`).

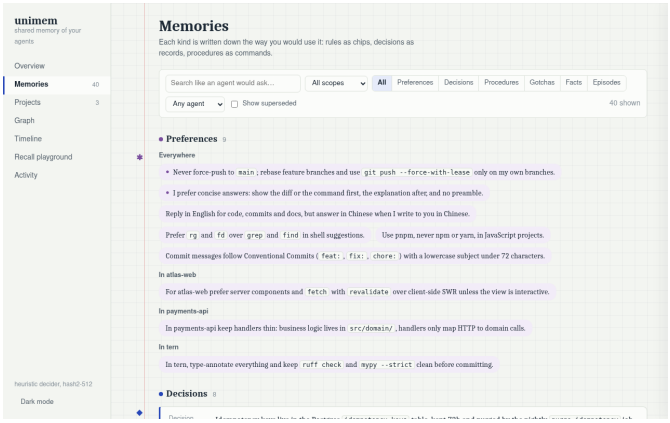
B Integration and configuration

```
unimem serve                                    # 127.0.0.1:4777
# Claude Code (API key and OAuth headers are forwarded)
export ANTHROPIC_BASE_URL=http://127.0.0.1:4777
# ~/.codex/config.toml
model_provider = "unimem"
[model_providers.unimem]
name = "unimem"
base_url = "http://127.0.0.1:4777/v1"
wire_api = "responses"
requires_openai_auth = true
# MCP-only mode (tools, no passive recall):
claude mcp add --transport http unimem http://127.0.0.1:4777/mcp
```

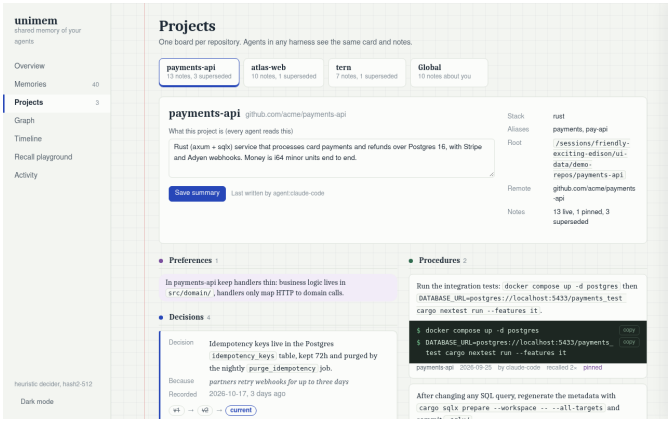
Table 10: Main configuration keys and defaults (`~/.unimem/config.toml`; every field optional).

section	key	default
`[inject]`	`recall_budget_ms`, `max_recall_notes`, `preamble_max_notes`, `max_continuations`	1500, 5, 8, 6
	`max_inject_tokens`, `max_preamble_tokens`, `max_tool_tokens`, `max_session_share`	1200, 1500, 3000, 0.05
	`tool_recall_every`, `tool_recall_max_tokens`, `flush_at`	5, 400, 0.7
	`budgets`, `dedup`, `tool_recall`, `correction_hints`	true
`[decider]`	`chain`	jev, llm, heuristic (jev only with `TYPESAFE_API_KEY`)
`[embedder]`	`kind`	hash (offline); openai; onnx
`[distill]`	`enabled`, `idle_secs`, `episode_ttl_days`, `min_rate_limit_remaining`, `chunk_chars`	false, 600, 30, 0.1, 24000

C Web UI



(a) Memories: per-kind views (rule chips, ADR-style decisions with supersede chains, procedures, gotchas).



(b) Projects: the editable project card every agent reads, and the repository's notes by kind.

Figure 5: The local web UI served at `/ui` (single embedded page, no external assets). Other views: overview, graph, timeline, recall playground and activity log. The UI has no authentication.

D Additional results

Table 11: heldout2 R@5 by category (out-of-sample; hash embedder; vector columns use `hash2`). *n* counts all queries of the category; scope_isolation R@5 is over its 18 answerable queries (4 are forbidden-only).

category	*n*	v0.1	v0.2	v0.3	silo	vector-top5	vector-top5-cwd	bm25-top5-cwd
single_hop	65	0.692	0.723	0.892	0.338	0.908	0.846	0.738
paraphrase	31	0.290	0.323	0.419	0.258	0.387	0.484	0.355
chinese	38	0.658	0.526	0.763	0.368	0.605	0.816	0.763
cross_project	19	0.211	0.158	0.368	0.158	0.579	0.053	0.053
temporal	24	0.500	0.500	0.667	0.208	0.625	0.667	0.708
causal	23	0.696	0.696	0.739	0.261	0.609	0.696	0.739
scope_isolation	22	0.722	0.667	0.556	0.056	0.500	0.667	0.611

Table 12: v0.3 in-sample (heldout) versus out-of-sample (heldout2): retrieval transferred, write-time screens did not.

	R@5	R@1	rejection recall	injection screen	duplicate recall	supersede recall	stale returned	scope leak
heldout (in-sample)	0.677	0.573	1.000	6/6	0.545	0.846	6.3%	0.0%
heldout2 (out-of-sample)	0.688	0.555	0.522	1/7	0.174	0.640	16.3%	2.5%

Table 13: LongMemEval$_S$ full set (419 questions, v0.4): session and turn index, plus the per-type turn R@10 that motivated the v0.5 refit.

system	sess. R@5	sess. R@10	turn R@5	turn R@10	multi-session	pref.	temporal	know.-update
bm25-okapi-ref (= paper BM25)	0.742	0.823	0.489	0.592	0.372	0.400	0.614	0.778
bm25 (stemmed)	0.859	0.924	0.654	0.754	0.587	0.500	0.756	0.931
e5-small	0.912	0.969	0.663	0.835	0.736	0.700	0.795	0.986
hybrid-rrf	0.921	0.971	0.721	0.838	0.711	0.733	0.827	0.986
unimem-prior-e5 (v0.4)	0.847	0.938	0.671	0.783	0.628	0.533	0.787	0.944
unimem-e5 (adaptive)	0.618	0.618	0.458	0.465	0.265	0.333	0.409	0.542

Table 14: v0.5 dev-split fit check (`prior_fit.json`; candidate pool = anchors; R@5 / R@10).

source (dev)	v0.4 prior	RRF only	v0.5 prior
LME session, e5	0.861 / 0.954	0.935 / 0.963	0.931 / 0.982
LME turn, e5	0.690 / 0.796	0.736 / 0.847	0.755 / 0.866
devmem, e5	0.810 / 0.899	0.832 / 0.894	0.838 / 0.899
LME turn, hash	0.694 / 0.778	0.639 / 0.722	0.694 / 0.768
heldout, hash	0.775 / 0.820	0.770 / 0.809	0.775 / 0.815